

기말고사

"Mini-Telegram"

학과: 소프트웨어융합학부

학번: 202155011

이름: 송영욱

1. 개요

이 프로젝트는 PyCryptodome을 이용한 암호화 기능(AES, RSA, 해시, PBE 등)과 Streamlit 기반 프로젝트입니다. 또한 간단한 블록체인 모듈을 통해 메시지 전송 기록을 블록으로 저장하고 채굴하는 시뮬레이션(간단 PoW)을 포함합니다. 주요 목적은 메시지의 기밀성, 무결성, 인증 개념을 시각적으로 이해하고 실습할 수 있도록 하는 것입니다. 배운만큼 파일도 나누어서 보기 좋게 구성하려고 노력했습니다. 아이디어는 교수님께서 수업중에 Telegram의 원리를 바탕으로 저의 방식대로 수업에 배운 암호화를 이용하여 똑같은 원리에 다른 암호화를 사용하여 작성해 보았습니다.

2. 구성

▶ [ui 파일]

ui.py

- Streamlit 애플리케이션 메인 파일. 탭별로 AES-CBC, SHA-512+AES, RSA 시뮬레이션, 고전 암호, AES-OFB, RSA 디지털 서명, Mini Telegram(채팅+블록체인)을 제공.

▶ [Crypto_lib 파일]

modren_crypto_lib.py

- AES, RSA, AES-GCM, AES-OFB, 디지털 서명 등 현대 암호 유틸 함수들.

classic_crypto.py

- 카이사르, 아핀, 비즈네르, 조합 암호 및 카이제곱 통계 함수 등 고전 암호 관련 유틸.

▶ [Util 파일]

crypto_utils.py

- PBE(PBKDF2-SHA512 + AES-GCM)를 이용한 개인키 암호화/복호화, AES-GCM 암호복호화 래퍼, DH 파라미터/키, 해시 유틸 등.

blockchain_utils.py

- 간단한 Block/Blockchain 구현(트랜잭션 저장, mine_block 으로 PoW 채굴).

3. 사용자 인터페이스 (UI/UX)

직관적인 다크 모드(Dark Mode) 인터페이스를 적용했습니다. 각 기능은 탭(Tab)으로 명확하게 구분하였고 동작 과정은 확장으로 제시하였습니다. 암호화에 대한 설명을 제시하였습니다.

4. 기술

대칭키 암호 (Symmetric-Key Ciphers)

1) AES-CBC (Cipher Block Chaining)

구현: 사용자가 키 크기를 (128bit, 192bit, 256bit)로 선택 할 수 있습니다. 새 랜덤 키 생성으로 대칭키를 생성합니다. 그리고 전송하고자하는 메시지를 입력합니다. 암호화를 누르면 AES의 과정과 암호화 결과를 보여줍니다. AES 복호화를 누르면 과정하고 메시지를 보여줍니다.

특징: 암호화 시 PKCS7 패딩, IV(초기화 벡터) 생성, 암호화 블록 체이닝 과정을 단계별로 설명하여 CBC 모드의 동작 원리를 시각적으로 보여줍니다

2) AES-OFB (Output Feedback)

구현: 블록 암호인 AES를 스트림 암호처럼 사용하는 OFB 모드를 합니다.

특징: 키를 랜덤으로 새로 생성하고 IV로 키 스트림(Keystream)을 생성하고, 이를 평문과 XOR하는 과정을 설명하여 스트림 암호의 핵심 원리를 제시합니다.

무결성 보장 암호 (Authenticated Encryption)

1) SHA-512 + AES

구현: 메시지의 기밀성(Confidentiality)과 무결성(Integrity)을 동시에 제공하는 E(H(M) || M) 방식을 구현했다. 사용자가 비밀번호를 입력하는 대신, 보안성이 높은 무작위 대칭키를 생성하여 사용하도록 변경합니다.

암호화 과정:

원본 메시지(M)에 대해 SHA-512 해시값 H(M)을 계산합니다.

H(M)과 M을 결합(H(M) || M)하여 페이로드를 생성합니다.

생성된 페이로드를 AES-CBC 모드로 암호화합니다.

복호화 및 검증 과정:

암호문을 복호화하여 H(M)과 M을 분리합니다.

분리된 M으로 다시 SHA-512 해시값 H'(M)을 계산합니다.

복호화된 H(M)과 새로 계산한 H'(M)을 비교하여 메시지 변조 여부를 확인합니다.

공개키 암호 (Public-Key Ciphers)

1) RSA 암호/복호화

구현: Alice → Bob 통신 시뮬레이션 탭에서 RSA 공개키 암호화를 구현했다. Alice는 Bob의 공개키로 메시지를 암호화하며, 이 암호문은 Bob의 개인키로만 복호화할 수 있다. 암호화 알고리즘을 AES_OFB, AES_CBC, SHA-512를 선택할 수 있다. 중간 패킷에 대한 정보를 보여주고 그것을 바탕으로 복호화를 진행한다.

KEY 생성: 랜덤으로 RSA키를 생성합니다

암호화 알고리즘을 선택: AES-CBC, AES-OFB, SHA-512 중 하나를 선택하고 랜덤으로 대칭키를 생성합니다

보낼 메시지를 입력합니다.

보내는 과정을 네트워크 패킷으로 JSON으로 보여줍니다.

bob이 복호화합니다.

2) RSA 디지털 서명

구현: 메시지의 무결성, 인증, 부인 방지를 보장하는 RSA 디지털 서명 기능을 별도의 탭으로 구현했습니다.

서명 과정: 송신자가 자신의 개인키를 사용하여 메시지의 SHA-512 해시값을 암호화 (서명)합니다.

검증 과정: 수신자가 송신자의 공개키를 사용하여 서명을 복호화하고, 함께 전송된 메시지의 해시값과 비교하여 서명의 유효성을 검증합니다.

고전 암호 (Classical Ciphers)

단순 치환 암호부터 다중 치환 암호까지 다양한 고전 암호를 구현하여 암호학의 기본 원리를 학습할 수 있도록 구성했습니다.

카이사르(Caesar) 암호: 알파벳을 일정한 거리만큼 밀어서 치환. 카이제곱을 통해 복호화합니다.

아핀(Affine) 암호: 일차함수 $(ax + b) \bmod m$ 을 이용한 치환 암호.

비즈네르(Vigenere) 암호: 키워드를 반복 사용하여 여러 개의 카이사르 암호를 적용하는 다중 치환 암호.

조합 암호: 카이사르 암호와 아핀 암호를 순차적으로 적용하여 보안성을 높인 방식.

Mini-Telegram

구현: Mini-Telegram 탭은 Diffie-Hellman 기반 세션키 유도과 AES-GCM에 의한 인증된 대칭 암호화를 이용하여 두 사용자(A, B) 간의 기밀·무결성 통신을 시뮬레이션합니다. 전송 실패 시 메시지는 로컬 블록체인에 트랜잭션으로 저장되어 즉시 채굴(시연용 PoW)됩니다. 개인키는 PBE(PBKDF2+AES-GCM)로 암호화되어 저장되며, 채팅 사용을 위해서는 패스워드로 개인키를 언락해야 합니다.

암호화 과정 (송신자 측)

1. 메시지 준비 및 무결성 해시 생성
 - 송신자는 먼저 평문에 대해 SHA-256 해시를 계산하여 무결성 검증용 해시를 준비합니다. (함수: `crypto.get_message_hash`)
2. DH 키 교환으로 세션키 유도
 - 송신자는 자신의 DH 개인키와 수신자의 DH 공개키를 사용해 공유 비밀을 계산하고, KDF(SHA-256)를 통해 32바이트 세션키를 파생합니다. (함수: `crypto.derive_dh_shared`)
 - AES-256-GCM으로 암호화 (AEAD)
 - 파생된 세션키로 AES-GCM을 수행하여 ciphertext, 인증 태그(tag), nonce를 생성합니다. (함수: `crypto.aes_encrypt`)
3. 전송 번들 구성 및 전송/저장
 - 번들에 sender, sender_dh_pub, nonce, ciphertext, tag, integrity_hash 등을 포함하여 TCP로 전송 시도(`send_via_tcp`). 전송 실패 시 블록체인 트랜잭션으로 저장하고 즉시 채굴(`blockchain.add_transaction()` + `mine_pending_transactions()`).

복호화 및 검증 과정 (수신자 측)

4. 번들 수신(혹은 블록체인에서 트랜잭션 조회)
 - DH 공개키를 이용해 동일 세션키 파생
 - 수신자는 자신의 DH 개인키와 번들에 포함된 송신자 공개키로 공유 비밀을 계산하여 동일한 세션키를 얻습니다. (함수: `crypto.derive_dh_shared`)
5. AES-GCM 복호화 및 태그 검증
 - 세션키로 ciphertext를 복호화하고 태그를 검증합니다. 태그 검증 실패 시 변조 또는 키 불일성으로 처리합니다. (함수: `crypto.aes_decrypt`)
6. 무결성 비교 (추가 확인)
 - 복호화된 메시지로 SHA-256을 재계산해 번들의 integrity_hash와 비교하여 최종 무결성 확인. 성공 시 메시지를 채팅 기록에 저장하고 로그에 처리 내역을 남깁니다.

부가 기능 / 구현 세부

- 키 관리: RSA 개인키는 PBKDF2(SHA-512)로 파생한 키로 AES-GCM 암호화되어 저장됩니다. 사용자는 패스워드로 개인키를 Unlock해야 채팅이 가능합니다.
(함수: `crypto.encrypt_private_key`, `crypto.decrypt_private_key`)
- 로그 및 원장: 전송/복호화/채굴 관련 단계별 로그를 `st.session_state.logs`에 남기고, 블록체인은 간단한 Block/Blockchain 클래스(`blockchain_utils.py`)로 구현되어 트랜잭션 저장·채굴을 시뮬레이션합니다.
- 네트워크: 로컬 TCP 릴레이(0.0.0.0:9632)를 띄워 번들을 수신하면 블록체인에 기록하거나 수신자에게 전달합니다.

[AI 사용부분 설명]

- AI는 전체적인 색상 조합, 폰트, 버튼, 탭 스타일 등 UI 디자인을 위한 CSS 코드 생성에 활용되었다.
- 사용자 경험을 개선하기 위해 각 기능에 맞는 아이콘(, ,  등)을 추천받아 버튼에 적용하였다.

```
if st.button("🔒 AES 암호화", key="encrypt_aes")
if st.button("🔓 AES 복호화", key="decrypt_aes"):
if st.button("🔑 새 랜덤 키 생성"):
```

※ ui.py 전체적인 구성 꾸미기

- 코드 리팩토링 및 구조화:
기존에 분리되어 있던 암호화 기능들을 하나의 Streamlit 애플리케이션으로 통합하고, 탭(Tab) 설명구조를 구성하는 과정에서 코드 구조에 대한 조언을 얻었습니다.

```
with tab2:
    st.header
    with st.expander:
        st.markdown("""
        """)
```

※ ui.py 383~389

- 각 암호화 알고리즘의 동작 원리를 사용자가 쉽게 이해할 수 있도록, 단계별 설명 텍스트와 `st.latex`를 이용한 수학 공식 표현을 추가하는 데 도움을 받았다. 또한 AES-CBC에서 구현 방식을 이미지로 보여주고 싶어서 사용한 `st.image`를 도움 받았습니다.

```
st.subheader("🔒 암호화 과정")
st.markdown(f"각 알파벳 문자를 알파벳 순서상  $\{shift\}$ 칸** 뒤로 이동시킵니다.")

example_char = next((c for c in text if c.isalpha()), 'A')
encrypted_char = caesar_encrypt(example_char, shift)
st.latex(f"E(x) = (x + \{shift\}) \pmod{26}")
st.markdown(f"**예시**:  $\{example\_char\} \rightarrow \{encrypted\_char\}$ ")
```

✖ ui.py 413~418

- 비즈네르 암호와 AES-CBC 부분을 구성 할 때 CBC는 방식을 물어보았습니다. 비즈네르의 경우 반복치환 과정에서 오류가 계속 떠서 물어 봤습니다.

```
for char in text:
    if char.isalpha():
        ascii_offset = ord('A') if char.isupper() else ord('a')
        x = ord(char) - ascii_offset
        encrypted = (a * x + b) % 26
        result += chr(encrypted + ascii_offset)
    else:
        result += char
```

✖ classic_crypto.py 69~76

- 암호/복호화 조합 부분에서 순서를 착각하여 오류가 나던 부분을 못찾고 있었는데 AI를 통해 실수를 잡았습니다.

```
def aes_ofb_decrypt(iv_b64: str, ciphertext_b64: str, key: bytes) -> str:
    iv = b64d(iv_b64)
    ciphertext = b64d(ciphertext_b64)
    cipher = AES.new(key, AES.MODE_OFB, iv=iv)
    plaintext = cipher.decrypt(ciphertext)
    return plaintext.decode("utf-8")
```

✖ modern_crypto_lib.py 98~103

- ui.py에서 Crypto_lib의 함수들을 import해오지 못해서 AI를 통해 해결했습니다.

```
import os
import sys
ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
```

※ ui.py 2~6

- 채팅을 구성하기 위해서 TCP를 사용해해서 메시지를 주고 받는 것을 하기 위해 AI의 도움을 받았습니다.

```
def send_via_tcp(bundle: dict, host: str = '127.0.0.1', port: int = 9632) -> bool:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect((host, port))
        s.sendall(json.dumps(bundle).encode('utf-8'))
        s.shutdown(socket.SHUT_WR)
        s.close()
        return True
    except Exception:
        return False
```

※ ui.py 108~117

- 메시지를 원장에 저장하는 과정에서 메시지가 자꾸 저장이 안되서 AI의 도움을 받았습니다.

```

def start_tcp_server(host: str, port: int, blockchain, logs_append):
    def handle_client(conn, addr):
        try:
            with conn:
                data = b""
                while True:
                    chunk = conn.recv(4096)
                    if not chunk:
                        break
                    data += chunk
                try:
                    msg = json.loads(data.decode('utf-8'))
                except Exception:
                    logs_append(f"[TCP] 잘못된 JSON 수신 from {addr}")
                    return
                blockchain.add_transaction(msg)
                new_block = blockchain.mine_pending_transactions()
                if new_block:
                    logs_append(f"[TCP] 블록 채굴 완료(Index {new_block.index})")
        except Exception as e:
            logs_append(f"[TCP] 클라이언트 처리 오류: {e}")

```

※ ui.py 63~83